

From the perspective of an **Executive Director (ED) on Nomura's Cash Equities Central Risk Book (CRB) desk**, managing overnight risk exposure, trading inventories, and pricing residual block risk require an accurate model of time-varying volatility clusters. When we internalize client order flow, a sudden market shock can trigger high-volatility regimes.

Understanding the mathematical constraints of a GARCH(1,1) process is vital. If the desk miscalculates volatility persistence, our transaction cost models, value-at-risk (VaR) limits, and portfolio optimization frameworks will fail, leading to mispriced risk and unexpected capital drag.

1. The Mathematical Deep Dive: Step-by-Painstaking-Step

Step 1: Formal Model Specification

The standard Generalized Autoregressive Conditional Heteroskedasticity, or **GARCH(1, 1)**, process decomposes asset returns into a deterministic conditional mean and a stochastic innovations component. We define the return r_t at time t as:

$$r_t = \mu + \epsilon_t$$

Where μ is the conditional mean (which we will treat as a constant or clean trend), and ϵ_t is the unexpected shock (residual return). This innovation term is structured as:

$$\epsilon_t = \sigma_t z_t$$

Here, z_t is a sequence of independent and identically distributed (i.i.d.) random variables acting as white noise, such that:

$$\mathbb{E}[z_t] = 0, \quad \text{Var}(z_t) = \mathbb{E}[z_t^2] = 1$$

The core of the **GARCH(1, 1)** framework is its recursive, conditional variance equation σ_t^2 , which depends on the immediate past squared residual and the immediate past conditional variance:

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

To ensure that the conditional variance remains strictly positive ($\sigma_t^2 > 0$) for all t , we must impose structural constraints on the parameter vector:

$$\omega > 0, \quad \alpha \geq 0, \quad \beta \geq 0$$

Step 2: Relating Innovations to Conditional Expectations

By definition, the conditional variance σ_t^2 is known given the filtration (information set) up to time $t - 1$, denoted as $\mathcal{F}_{t-1}$. Let us evaluate the conditional expectation of the squared shock ϵ_{t-1}^2 :

$$\mathbb{E}[\epsilon_{t-1}^2 \mid \mathcal{F}_{t-2}] = \mathbb{E}[\sigma_{t-1}^2 z_{t-1}^2 \mid \mathcal{F}_{t-2}]$$

Since σ_{t-1}^2 is fully determined by the past information set $\mathcal{F}_{t-2}$, it can be factored out of the conditional expectation:

$$\mathbb{E}[\epsilon_{t-1}^2 \mid \mathcal{F}_{t-2}] = \sigma_{t-1}^2 \mathbb{E}[z_{t-1}^2 \mid \mathcal{F}_{t-2}]$$

Because z_{t-1} is independent of past information and has a variance of 1, this simplifies cleanly to:

$$\mathbb{E}[\epsilon_{t-1}^2 \mid \mathcal{F}_{t-2}] = \sigma_{t-1}^2 \cdot 1 = \sigma_{t-1}^2$$

Applying the **Law of Total Expectation** (the tower property), the unconditional expectation of the squared innovation matches the unconditional expectation of the conditional variance:

$$\mathbb{E}[\epsilon_{t-1}^2] = \mathbb{E}[\mathbb{E}[\epsilon_{t-1}^2 \mid \mathcal{F}_{t-2}]] = \mathbb{E}[\sigma_{t-1}^2]$$

Step 3: Deriving the Unconditional (Long-Run Steady-State) Variance

If the process is covariance stationary, its statistical properties do not shift over time. This implies that the unconditional long-run variance reaches a constant, steady-state plateau:

$$\mathbb{E}[\sigma_t^2] = \mathbb{E}[\sigma_{t-1}^2] = \bar{\sigma}^2$$

Now, we take the unconditional expectation of both sides of the recursive variance equation:

$$\mathbb{E}[\sigma_t^2] = \mathbb{E}[\omega + \alpha\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2]$$

By the linearity of expectations:

$$\mathbb{E}[\sigma_t^2] = \omega + \alpha\mathbb{E}[\epsilon_{t-1}^2] + \beta\mathbb{E}[\sigma_{t-1}^2]$$

Substituting our findings from Step 2 ($\mathbb{E}[\epsilon_{t-1}^2] = \mathbb{E}[\sigma_{t-1}^2] = \bar{\sigma}^2$) into this expression yields:

$$\bar{\sigma}^2 = \omega + \alpha\bar{\sigma}^2 + \beta\bar{\sigma}^2$$

Isolating the steady-state variance $\bar{\sigma}^2$:

$$\bar{\sigma}^2 - \alpha\bar{\sigma}^2 - \beta\bar{\sigma}^2 = \omega$$

$$\bar{\sigma}^2(1 - \alpha - \beta) = \omega$$

$$\boxed{\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \beta}}$$

Step 4: Deducing the Covariance Stationarity Frontier

For this long-run unconditional variance to represent a valid, finite, and positive physical constraint, the denominator must remain strictly positive. Therefore, the parameter boundary condition for covariance stationarity is:

$$\alpha + \beta < 1$$

If $\alpha + \beta > 1$, the denominator turns negative, rendering the formula invalid. Mechanically, this means that the unconditional variance explodes to infinity over long horizons, breaking stationarity.

Step 5: Deconstructing Volatility Persistence and Shock Decay Kinetics

Persistence measures how long a historical volatility shock impacts future variance forecasts. To observe this decay rate, let us define a variance shock innovation term as $v_t = \epsilon_t^2 - \sigma_t^2$. Notice that $\mathbb{E}[v_t \mid \mathcal{F}_{t-1}] = 0$, meaning it behaves as a martingale difference sequence.

We can rewrite our conditional variance equation by substituting $\epsilon_{t-1}^2 = \sigma_{t-1}^2 + v_{t-1}$:

$$\sigma_t^2 = \omega + \alpha(\sigma_{t-1}^2 + v_{t-1}) + \beta\sigma_{t-1}^2$$

$$\sigma_t^2 = \omega + (\alpha + \beta)\sigma_{t-1}^2 + \alpha v_{t-1}$$

This highlights that the variance process maps structurally to an **ARMA(1, 1)** model for ϵ_t^2 . By recursively projecting this equation k steps into the future, the expected conditional variance forecast satisfies:

$$\mathbb{E}[\sigma_{t+k}^2 \mid \mathcal{F}_t] - \bar{\sigma}^2 = (\alpha + \beta)^k (\sigma_{t+1}^2 - \bar{\sigma}^2)$$

This expression shows that the deviation of current volatility from its long-run average decays exponentially at a rate governed by the magnitude of $(\alpha + \beta)$.

Step 6: Limiting Case — Integrated GARCH (IGARCH)

When the persistence metric hits the unit-root limit, $\alpha + \beta = 1$, the model simplifies to **Integrated GARCH (IGARCH)**. Substituting $\beta = 1 - \alpha$ into the conditional variance equation:

$$\sigma_t^2 = \omega + \alpha\epsilon_{t-1}^2 + (1 - \alpha)\sigma_{t-1}^2$$

In an **IGARCH** regime:

- The long-run unconditional variance $\bar{\sigma}^2 = \frac{\omega}{0} \rightarrow \infty$. The process loses a well-defined long-run mean.
- Current shocks never decay; instead, they alter the baseline volatility map permanently across all future horizons $((\alpha + \beta)^k = 1^k = 1)$.

2. Application of Theory and Production-Grade Implementation

In a production execution architecture for a Central Risk Book, we do not feed raw asset returns directly into basic statistical loops. We must implement a robust Maximum Likelihood Estimation (**MLE**) framework to calibrate our model parameters (ω, α, β) . This script handles parameter boundaries via optimization constraints, penalizes non-stationary paths, and computes long-run unconditional targets.

Production-Grade Python GARCH Engine

```
import numpy as np
import pandas as pd
from scipy.optimize import minimize

class CRBGarchEngine:
    """
    Production-grade GARCH(1,1) parameter calibration and forecasting engine
    designed for Nomura's Central Risk Book overnight exposure management.
    """
    def __init__(self):
        self.omega = None
        self.alpha = None
        self.beta = None
        self.unconditional_vol = None
```

```

def _negative_log_likelihood(self, params: np.ndarray, returns: np.ndarray) ->
float:
    """
    Computes the negative log-likelihood of a GARCH(1,1) process.
    """
    omega, alpha, beta = params
    n = len(returns)

    # Initialize array for conditional variances
    # A common production initialization is setting the first variance to the
sample variance
    sample_var = np.var(returns)
    cond_vars = np.zeros(n)
    cond_vars[0] = sample_var

    # Guard against zero or negative values in the variance loop
    eps = 1e-6
    log_likelihood = 0.0

    for t in range(1, n):
        # GARCH(1,1) recursive formula: sigma_t^2 = omega + alpha * eps_{t-
1}^2 + beta * sigma_{t-1}^2
        # Here, returns are assumed to be mean-centered residuals (mu=0
baseline)
        cond_vars[t] = omega + alpha * (returns[t-1] ** 2) + beta *
cond_vars[t-1]

        # Bound conditional variance away from zero to avoid log(0) errors
        if cond_vars[t] < eps:
            cond_vars[t] = eps

        # Log-Likelihood contribution for a normal distribution component
        log_likelihood += -0.5 * np.log(2 * np.pi) - 0.5 *
np.log(cond_vars[t]) - 0.5 * (returns[t] ** 2) / cond_vars[t]

    return -log_likelihood

def fit(self, returns: np.ndarray) -> dict:
    """
    Calibrates GARCH(1,1) parameters using constrained Maximum Likelihood
Estimation.
    """
    # Center the returns to focus on volatility innovations
    mean_centered_returns = returns - np.mean(returns)
    sample_var = np.var(mean_centered_returns)

    # Initial guess: typical liquid equity profiles (alpha ~ 0.05, beta ~
0.90)
    initial_guess = [sample_var * 0.05, 0.05, 0.90]

    # Bounds to ensure non-negativity and strictly positive omega
    bounds = ((1e-6, None), (0.0, 1.0), (0.0, 1.0))

    # Covariance stationarity constraint: alpha + beta <= 0.999 (to prevent

```

```

unconstrained IGARCH explosion)
def stationarity_constraint(params):
    return 0.999 - (params[1] + params[2])

constraints = {'type': 'ineq', 'fun': stationarity_constraint}

# Optimization execution
result = minimize(
    self._negative_log_likelihood,
    initial_guess,
    args=(mean_centered_returns,),
    method='SLSQP',
    bounds=bounds,
    constraints=constraints
)

if not result.success:
    raise ValueError(f"GARCH optimization calibration failed:
{result.message}")

self.omega, self.alpha, self.beta = result.x

# Compute unconditional variance: omega / (1 - alpha - beta)
unconditional_var = self.omega / (1.0 - (self.alpha + self.beta))
self.unconditional_vol = np.sqrt(unconditional_var)

return {
    "omega": self.omega,
    "alpha": self.alpha,
    "beta": self.beta,
    "persistence": self.alpha + self.beta,
    "unconditional_vol": self.unconditional_vol
}

def forecast_variance(self, returns: np.ndarray, horizon: int) -> np.ndarray:
    """
    Projects conditional variance across a forward horizon using the
    calibrated GARCH profile.
    """
    if self.omega is None:
        raise ValueError("Engine must be fitted before running forecasts.")

    mean_centered_returns = returns - np.mean(returns)

    # Calculate the final conditional variance entry from past historical data
    n = len(mean_centered_returns)
    last_cond_var = np.var(mean_centered_returns)
    for t in range(1, n):
        last_cond_var = self.omega + self.alpha * (mean_centered_returns[t-1]
** 2) + self.beta * last_cond_var

    # Allocate array for forward projections
    forecasts = np.zeros(horizon)
    persistence = self.alpha + self.beta

```

```

        unconditional_var = self.unconditional_vol ** 2

        # Step 1 projection uses the last available real data point
        forecasts[0] = self.omega + self.alpha * (mean_centered_returns[-1] ** 2)
        + self.beta * last_cond_var

        # Steps 2 to horizon use the iterative conditional expectation formulation
        for k in range(1, horizon):
            # E[sigma_{t+k}^2] = long_run_var + persistence * (E[sigma_{t+k-1}^2]
            - long_run_var)
            forecasts[k] = unconditional_var + persistence * (forecasts[k-1] -
            unconditional_var)

        return forecasts

```

3. Executive Director's Follow-up Questions & Production Fixes

Follow-Up 1: "Asset returns frequently exhibit asymmetric volatility responses—meaning negative return shocks trigger a larger jump in volatility than positive shocks of equal magnitude (the leverage effect). Standard GARCH completely ignores this sign asymmetry. How do you resolve this mathematically and programmatically?"

- **The Quantitative Fix:** Standard GARCH models use squared returns (ϵ_{t-1}^2), which strips out the directional sign of the shock. To capture this asymmetry, we can upgrade our system to an **Asymmetric GARCH** framework, such as the **Glosten-Jagannathan-Runkle GARCH (GJR-GARCH)** model. The conditional variance equation expands to:

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \gamma \epsilon_{t-1}^2 I_{t-1} + \beta \sigma_{t-1}^2$$

- Where I_{t-1} is an indicator function that flags negative returns:

$$I_{t-1} = \begin{cases} 1 & \text{if } \epsilon_{t-1} < 0 \\ 0 & \text{if } \epsilon_{t-1} \geq 0 \end{cases}$$

- **The Structural Stationarity Adjustment:** Under this asymmetric structure, assuming the distribution of ϵ_t is symmetric around zero, the new covariance stationarity condition becomes:

$$\alpha + \frac{\gamma}{2} + \beta < 1$$

- **The Code Adjustment:** To update our production engine, we insert the indicator function into the negative log-likelihood optimization loop:

```

def _gjr_negative_log_likelihood(self, params: np.ndarray, returns: np.ndarray) ->
float:
    omega, alpha, gamma, beta = params
    n = len(returns)
    cond_vars = np.zeros(n)
    cond_vars[0] = np.var(returns)

```

```

log_likelihood = 0.0
eps = 1e-6

for t in range(1, n):
    # Indicator variable captures the sign of the previous period's shock
    indicator = 1.0 if returns[t-1] < 0.0 else 0.0

    # GJR-GARCH step
    cond_vars[t] = (omega +
                    alpha * (returns[t-1] ** 2) +
                    gamma * (returns[t-1] ** 2) * indicator +
                    beta * cond_vars[t-1])

    if cond_vars[t] < eps:
        cond_vars[t] = eps

    log_likelihood += -0.5 * np.log(2 * np.pi) - 0.5 * np.log(cond_vars[t]) -
0.5 * (returns[t] ** 2) / cond_vars[t]

return -log_likelihood

```

Follow-Up 2: "If your calibrated model points to an IGARCH regime ($\alpha + \beta = 1$), what are the exact practical implications for the CRB when pricing a multi-week variance swap or a long-dated OTC option?"

- **The Quantitative Fix:** In an IGARCH regime, a volatility shock does not decay. This permanently shifts the baseline for future volatility expectations.
- **Impact on Long-Dated Instruments:** For standard stationary models, forward-starting variance forecasts revert back to the long-run average ($\bar{\sigma}^2$) over time. This creates a square-root term-structure effect where long-dated options show lower sensitivity to daily spot volatility spikes.
- Under IGARCH, however, the variance forecast profile remains flat and elevated across the entire horizon:

$$\mathbb{E}[\sigma_{t+k}^2 \mid \mathcal{F}_t] = \sigma_{t+1}^2 + (k - 1)\omega$$

- When pricing variance swaps or long-dated options for a client, assuming IGARCH means that a volatility spike today amplifies risk expectations across the entire life of the trade. If this spike is temporary, using an unadjusted IGARCH profile will overprice the hedge, causing us to lose the client's business to competitors. Conversely, if the shift is structural and we misprice it using a fast-reverting model, the CRB will be exposed to unhedged tail risk.